

Examiner — Complete System Architecture & Workflow

1. Project Overview

Project Name

Examiner

Project Type

AI-Powered Virtual Interview Platform

Development Category

- Full Stack AI Application
 - Conversational AI System
 - LLM-Powered Platform
 - Generative AI Application
 - Multimodal AI System
-

2. Problem Statement

Existing Problem

Students prepare for interviews using:

- YouTube videos
- Notes
- PDFs
- Static MCQ platforms
- Traditional coding websites

But these methods have several limitations.

Problems in Existing Systems

1. No Real Interview Experience

Most platforms only provide:

- MCQs
- static questions
- theory content

They do not simulate real interviewer interaction.

2. No Communication Practice

Students may know concepts but fail to:

- explain properly
 - communicate confidently
 - answer dynamically
-

3. No Confidence Analysis

Current systems cannot analyze:

- nervousness
 - hesitation
 - speaking confidence
 - eye contact
 - behavior
-

4. No Personalized Interview Flow

Most interview preparation systems use:

- fixed questions
- static test patterns

They cannot dynamically adapt based on candidate answers.

5. Lack of Realistic Environment

Students do not experience:

- pressure of real interviews

- human-like interaction
- conversational flow

Because of this, many students fail real interviews despite having technical knowledge.

3. Proposed Solution

Examiner

Examiner is an AI-powered conversational interview platform that simulates realistic technical interviews using:

- Generative AI
 - NLP
 - Speech Processing
 - Computer Vision
 - Real-time Interaction
-

Core Idea

The system behaves like a professional interviewer.

It:

- asks interview questions
 - evaluates answers
 - generates follow-up questions
 - analyzes communication
 - measures confidence
 - generates performance reports
-

Main Objective

To provide students with:

- realistic interview experience
 - technical preparation
 - communication improvement
 - confidence building
 - AI-based feedback
-

4. System Workflow

Complete User Flow

```
graph TD; A[User Opens Examiner] --> B[Animated AI Interviewer Appears]; B --> C[User Selects Subject]; C --> D[Interview Starts]; D --> E[AI Asks Question]; E --> F[Candidate Answers]; F --> G[AI Evaluates Response]; G --> H[AI Generates Follow-Up Question]; H --> I[Interview Continues]; I --> J[Final Report Generated];
```

5. Actual Working of Examiner

Step 1 — Landing Page

When user opens the application:

System displays:

- animated AI interviewer
- subject input field
- interview settings

Example:

```
Subject: Java
Difficulty: Medium
Interview Type: Technical
```

Step 2 — Interview Initialization

User clicks:

Start Interview

Backend creates:

- interview session
 - user context
 - AI prompt context
-

Step 3 — AI Interviewer Activation

Backend sends prompt to Gemini API.

Example Prompt:

Act as a professional Java interviewer.
Ask one technical question at a time.
Evaluate answers professionally.

Gemini generates first question.

Example:

What is JVM?

Step 4 — Candidate Response

Candidate answers using:

- text OR
 - voice
-

Step 5 — NLP & AI Processing

The system processes:

Text Analysis

Gemini analyzes:

- correctness
 - technical depth
 - communication quality
-

Speech Analysis

Whisper converts speech → text.

Librosa analyzes:

- confidence
 - pauses
 - voice stability
-

Computer Vision Analysis

OpenCV + MediaPipe analyze:

- eye contact
- facial movement
- posture
- attention

DeepFace analyzes:

- emotion
 - nervousness
 - stress
-

Step 6 — Dynamic Follow-Up Question

Based on answer quality:

If answer is weak

AI asks easier/follow-up questions.

Example:

Can you explain bytecode in more detail?

If answer is strong

AI increases difficulty.

Example:

Explain JVM memory architecture.

Step 7 — Final Report Generation

At interview end:

System generates:

- technical score
- communication score
- confidence score
- strengths
- weak areas
- improvement suggestions

6. Communication Between Components

Overall Communication Architecture

```
Frontend
  ↓
Spring Boot Backend
  ↓
Python ML Service
  ↓
Gemini API + NLP + CV Models
```

Detailed Communication Flow

FRONTEND → BACKEND

Frontend sends:

- subject
- user answer
- voice input
- webcam frames

Backend receives and manages:

- interview state
 - sessions
 - authentication
 - orchestration
-

BACKEND → GEMINI API

Backend sends:

- interview prompts
- candidate answers
- conversation history

Gemini returns:

- questions
 - evaluations
 - feedback
-

BACKEND → ML SERVICE

Backend sends:

- voice/audio
- webcam data
- candidate responses

ML Service performs:

- NLP analysis
- confidence analysis
- emotion detection

ML SERVICE → BACKEND

ML service returns:

- confidence score
- eye contact score
- emotion analysis
- communication metrics

BACKEND → FRONTEND

Backend sends:

- AI questions
- live responses
- interview feedback
- final report

7. Required Project Folders

Initial MVP Structure

```
Examiner/  
|  
├── Examiner-frontend/  
└── Examiner-backend/
```

This is enough for:

- text interview
- Gemini integration
- AI questioning
- feedback system

Advanced Architecture Structure

When adding:

- NLP
- OpenCV

- Speech AI
- Confidence Analysis

Then use:

```
Examiner/  
|  
├─ Examiner-frontend/  
├─ Examiner-backend/  
└─ Examiner-ml/
```

8. Responsibilities of Each Folder

A. Examiner-frontend

Purpose

User Interface Layer

Technologies

- React
 - Tailwind CSS
 - Framer Motion
 - React Three Fiber
-

Responsibilities

Frontend handles:

- landing page
 - animated interviewer
 - user interaction
 - voice recording
 - webcam access
 - interview UI
 - final report dashboard
-

B. Examiner-backend

Purpose

Main System Controller

Technologies

- Spring Boot
 - JWT
 - MongoDB/PostgreSQL
-

Responsibilities

Backend handles:

- APIs
 - authentication
 - interview sessions
 - Gemini communication
 - report generation
 - orchestration
 - frontend communication
-

C. Examiner-ml

Purpose

AI/NLP/Computer Vision Processing Layer

Technologies

- Python
- FastAPI
- OpenCV
- MediaPipe
- Whisper
- DeepFace
- Librosa
- spaCy

Responsibilities

ML service handles:

- speech processing
- emotion detection
- eye tracking
- confidence analysis
- NLP analysis
- behavioral analytics

9. AI Models Used in Examiner

Model/Tool	Purpose
Gemini API	Interview intelligence
Whisper	Speech-to-text
Coqui TTS	AI voice generation
OpenCV	Webcam analysis
MediaPipe	Eye/head tracking
DeepFace	Emotion detection
Librosa	Confidence analysis
spaCy	NLP processing

10. Technology Stack

Frontend

- React
- Tailwind CSS
- Framer Motion
- React Three Fiber

Backend

- Spring Boot
- Spring Security

- JWT
 - WebSocket
-

Database

- MongoDB OR
 - PostgreSQL
-

AI/ML

- Gemini API
 - Python FastAPI
 - OpenCV
 - Whisper
 - DeepFace
 - MediaPipe
-

11. Development Phases

PHASE 1 — MVP

Build:

- landing page
- subject input
- Gemini integration
- AI questions
- text interview
- final report

Folders required:

frontend + backend

PHASE 2 — Voice AI

Add:

- speech-to-text

- AI voice response
- real-time interaction

Folders required:

frontend + backend + ml

PHASE 3 — AI Behavioral Analysis

Add:

- OpenCV
- emotion detection
- confidence analysis
- eye tracking

Folders required:

frontend + backend + ml

12. Final System Architecture

```
Candidate
  ↓
Frontend (React)
  ↓
Spring Boot Backend
  ↓
Gemini API
  ↓
Python ML Service
  ↓
OpenCV + NLP + Speech Models
```

13. Innovation of Examiner

Unlike traditional platforms:

Examiner provides:

✓ Real-time AI interview ✓ Dynamic questioning ✓ Conversational interaction ✓ Communication analysis ✓ Confidence analysis ✓ Emotion detection ✓ Personalized feedback ✓ Realistic interviewer experience

14. Final Conclusion

Examiner is a Full Stack LLM-powered conversational AI platform designed to simulate realistic technical interviews using Generative AI, NLP, Speech Processing, and Computer Vision technologies.

The system combines:

- AI interaction
- voice intelligence
- behavioral analysis
- dynamic interview generation
- full stack architecture

to create an advanced AI-based interview preparation experience.